C554-Midterm Report / Evidence Log

Part 2 Docker Compose Inside A Linux VM

2. Start the service and demonstrate successful requests to both endpoints from inside VM

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (master)
$ multipass shell cs454-vm
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-34-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Tue Oct 20 15:47:55 CDT 2026

  System load:             0.0
  Usage of /:              60.9% of 3.70GB
  Memory usage:            22%
  Swap usage:              0%
  Processes:               106
  Users logged in:         0
  IPv4 address for enp0s3: 10.0.2.15
  IPv4 address for enp0s8: 192.168.1.215
  IPv6 address for enp0s8: 2605:a601:a81b:fe00::4
  IPv6 address for enp0s8: 2605:a601:a81b:fe00:5054:ff:fe86:6328


Expanded Security Maintenance for Applications is not enabled.

52 updates can be applied immediately.
46 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


Last login: Tue Oct 20 15:47:55 2026 from 10.0.2.2
ubuntu@cs454-vm:~$ SERVICE_NAME=vm-service MESSAGE="Hello from a VM environment" node app.js
Listening on port 8080
```

Successfully created a Virtual Machine and started the service while inside

```
ubuntu@cs454-vm:~$ curl http://192.168.1.215:8080/health
{"status":"ok"}ubuntu@cs454-vm:~$
```

```
ubuntu@cs454-vm:~$ curl http://192.168.1.215:8080
{"service":"vm-service","message":"Hello from a VM environment","hostname":"cs454-vm","requestCount":8}
```

Successfully requested both endpoints from inside the virtual machine

3. Demonstrate a request from the host computer to the service running inside the VM.

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (master)
$ curl http://192.168.1.215:8080
{"service":"vm-service","message":"Hello from a VM environment","hostname":"cs454-vm","requestCount":1}
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (master)
$ curl http://192.168.1.215:8080/health
{"status":"ok"}
```

Successfully requested both endpoints from outside of the virtual machine (host computer)

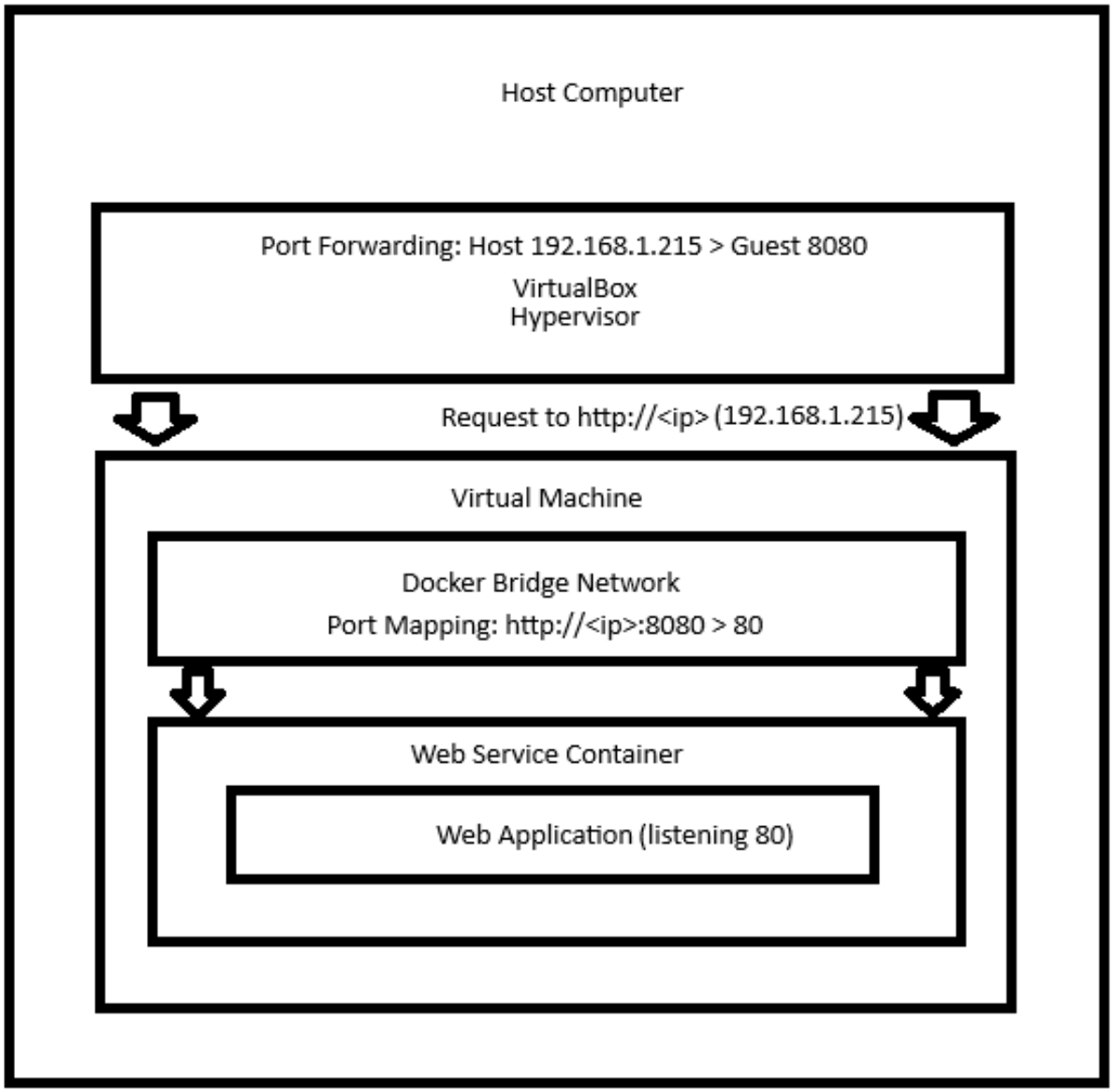
4. Show the service status and container logs

```
ubuntu@cs454-vm:~$ SERVICE_NAME=vm-service MESSAGE="Hello from a VM environment" node app.js
Listening on port 8080
{"method":"GET","path":"/","status":200}
{"method":"GET","path":"/health","status":200}
```

Service status and container logs from inside the VM

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (master)
$ curl http://192.168.1.215:8080
{"service":"vm-service","message":"Hello from a VM environment","hostname":"cs454-vm","requestCount":1}
```

After restarting the service, the request count restarts.



Part 3 Kubernetes Deployment

1. Create a deployment using the same application image, configured with two replicas.

In Repository

2. Create a service that selects the application Pods and exposes their HTTP port within the cluster.

In Repository

3. Demonstrate successful requests through the Service. A documented local access method such as port forwarding is acceptable.

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl port-forward svc/web-service 8080:80
Forwarding from 127.0.0.1:8080 -> 3000
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
```

Request logs and port forwarding so requests are directed to only one Pod

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ curl http://localhost:8080
{"service":"kubernetes","message":"Hello from kubernetes","hostname":"cs454-service-7d77c466d4-j24dw","requestCount":1}
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ curl http://localhost:8080
{"service":"kubernetes","message":"Hello from kubernetes","hostname":"cs454-service-7d77c466d4-j24dw","requestCount":2}
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ curl http://localhost:8080
{"service":"kubernetes","message":"Hello from kubernetes","hostname":"cs454-service-7d77c466d4-j24dw","requestCount":3}
```

In a separate terminal, multiple requests were made.

4. Show the Deployment, Pods, and Service, including evidence that two replicas are ready.

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl get deployments
NAME            READY   UP-TO-DATE   AVAILABLE   AGE
cs454-service   2/2     2            2           15m

tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl get pods -o wide
NAME                             READY   STATUS    RESTARTS   AGE   IP          NODE                NOMINATED NODE   READINESS GATES
cs454-service-7d77c466d4-j24dw   1/1     Running   0          15m   10.42.1.5   k3d-cs454-agent-1   <none>           <none>
cs454-service-7d77c466d4-km85t   1/1     Running   0          15m   10.42.2.4   k3d-cs454-server-0   <none>           <none>

tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl get services
NAME            TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
cs454-service   ClusterIP   10.43.104.112   <none>        80/TCP    16m
kubernetes      ClusterIP   10.43.0.1       <none>        443/TCP   17m
web-service     ClusterIP   10.43.222.169   <none>        80/TCP    16m

tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl get ingress
NAME            CLASS     HOSTS   ADDRESS                                 PORTS   AGE
cs454-service   traefik   *       172.21.0.2,172.21.0.3,172.21.0.4   80      16m
```

Deployments, Pods, and Service statistics

5. Delete one application Pod while Deployment still specifies two replicas. Observe and record the replacement Pod and the return to two ready replicas.

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl get pods
NAME                              READY   STATUS    RESTARTS   AGE
cs454-service-7d77c466d4-j24dw   1/1     Running   0          27m
cs454-service-7d77c466d4-km85t   1/1     Running   0          27m

tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl delete pod cs454-service-7d77c466d4-j24dw
pod "cs454-service-7d77c466d4-j24dw" deleted from default namespace

tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl get pods -w
NAME                              READY   STATUS    RESTARTS   AGE
cs454-service-7d77c466d4-42k42   1/1     Running   0          5s
cs454-service-7d77c466d4-km85t   1/1     Running   0          28m
```

After deleting the Pod, a replacement Pod automatically started.

Explain why Kubernetes creates a replacement Pod. Compare this behavior with what you observe when restarting the Compose Service.

Kuberenetes creates a replacement pod to maintain whatever state is declared by the controller. The controller doesn't execute it directly, but it sets the rules that must be maintained. The replacement pod completely destroys the initial pod and acts as a brand new entity. The Docker Compose restart, on the other hand, reuses the container's configuration and preserves any local container changes.

You do not need to prove round-robin traffic distribution. Some access methods, including port forwarding, can keep requests directed to a single Pod. Explain what your chosen access method actually demonstrates.

The port forwarding implemented is used to direct traffic to a singular pod. It creates a secure tunnel so that every HTTP request is forwarded to a singular pod, which allows for easier container and traffic management.

Part 4 Controlled Troubleshooting

1. The baseline and the exact configuration change that introduced the failure.

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl patch svc web-service -p '{"spec":{"selector":{"app":"web-frontend-broken"}}}'
```

The command used to implement the breaking change

2. The symptom observed by a client.

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ curl http://localhost:8080
no available server
```

The errors after attempting requests after breaking change

3. At least two relevant observations, such as Pod readiness, Pod labels, the Service selector, EndpointSlices, forwarding errors, or application logs.

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl port-forward svc/web-service 8080:80
error: timed out waiting for the condition
```

The following forwarding error occurred after breaking change

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl get endpointslices -l kubernetes.io/service-name=web-service
NAME                ADDRESSTYPE   PORTS     ENDPOINTS   AGE
web-service-sggsx   IPv4          <unset>   <unset>     17m
```

The endpoints list returned completely empty after the breaking change

In Kubernetes, the controller queries for Pods that match the service's selector. Since the selector was changed, the controller couldn't find any matching Pods. A healthy application indicates that the code is running, but it doesn't guarantee that the internal network is working properly, meaning we can have a running application with a broken network pipeline.

Part 5 - OpenStack Deployment Analysis

Resources and Choices

To deploy this workload on an instructor-provided OpenStack cloud, we need to define a few resources. The resources that may be supplied already by the cloud administrator would be the base operating system image, whether it is Ubuntu or any other Linux distribution, and the external network. We would create the subnet, floating IP, virtual router, and security group. Ubuntu-22.04-LTS-Clouding could be used as the Image, since it is one of the more common Linux cloud images. With a workload of this size, we don't need a large VM. A small to medium VM size, whether it's m1.small or m1.medium flavor with one to two vCPUs, 20 GBs of disk space, and two to four GB of RAM, would suffice. A modest VM size is justified because not only does the application consist of microservices that are lightweight, but they are also packaged on Docker containers instead of virtual machines. This allows the application to avoid excessive resource consumption and avoid any heavy memory expenses. Additionally, we could use developer-ssh-key as the key pair, which is a generated key pair used to enable secure CLI access. interal-overlay-new as the network and internal-overlay-subnet as its subnet can be used to allocate private IPs. tenant-gateway-router, which is used with the internal-overlay-subnet, will act as the router connection. Next, app-ingress-secgroup, a virtual firewall, could be used to only accept permitted incoming ports. Lastly, we could have dynamically allocated IPv4 addresses from the network pool. A persistent block storage is not required unless the architecture requires a database of some kind, like PostgreSQL, that needs to contain persistent data across restarts. However, this specific workload does not require any sort of database persistence, so it is unnecessary with its current requirements.

Service Responsibilities

Keystone is used to authenticate user credentials. It ultimately is used to validate permissions for the incoming API requests. Glance, acting as an asset catalog, manages the image registry available to the cloud. Its sole purpose is to provide metadata of chosen images. Neutron, the networking service, is responsible for handling connectivity by dynamically building a network topology. It creates the virtual switches, maps public and private IPs, and sets up virtual routers to appropriately authenticate a user. When selecting an image, Nova queries Glance for the availability of the chosen image. If available, Glance returns the raw storage blocks to the hypervisor host. After selecting an image, Nova evaluates resources across nodes in the cluster, finds a compatible host, builds a virtual desk image from the source file that was retrieved by Glance and boots the new VM instance. Once the instance has been created, Neutron allocates a MAC address and a private IP address from the DHCP pool. It, then, links the VM to your private network and virtual router. Once the virtual machine reaches the active state, Docker Compose takes over inside the guest OS. Docker Compose will continue to operate completely isolated from the OpenStack cloud by solely focusing on application execution via the

docker-compose.yml file. Although this specific deployment does not require database persistence, Cinder could be used to support a database container to do just that. It could be used to maintain persistent data that would be independent of the virtual machine's status.

Request Path
After the external client initiates an HTTP request through the floating IP, the packet arrives at the provider network and hits the interface of the Neutron Router. After performing DNAT, Destination Network Address Translation, it remaps the floating IP to a fixed IP that can be used on the virtual machine. The router, then, forwards the packet to the specific hypervisor that is hosting the virtual machine. The OpenStack Security Group then checks the retrieved packet's destination port and protocols before allowing it to reach the VM's network card. This is different from SSH access or optional ICMP/ping access because it only allows for web traffic to hit the API layer of the application. SSH access is typically reserved for administrative access and ICMP/ping access just tests the visibility of the network layer. The Docker container, then, manages the incoming traffic by forwarding it to the container's application port. Lastly, the application server processes the HTTP request and responds back.

Failure Analysis
If the external client cannot reach the service, we must check the connectivity of the virtual machine. First, we should check to see if there is an ICMP response by pinging the floating IP. If this fails, the problem is either a routing or a security group issue. Second, we need to check for SSH access. If the SSH remotes in successfully, the problem would likely be a security group issue rather than a cloud network issue. Next, we should check the firewall by reviewing the security group configurations of the specific VM's network. This could be done by using OpenStack CLI, we would expect there to be permission for Ingress traffic over TCP. If this is not defined, then the packets could be dropping prematurely. Another area we need to check is the container statuses and whether or not they are operating as expected. We must check to see if all required containers are up and verify that the Docker virtual bridge network is mapping ports as expected. Lastly, we should check the application logs for obvious errors, like exceptions, timeouts, or missing dependencies. After running through all of these troubleshooting techniques, the cause of the problem should be relatively clear.

Operational Complexity
Operating an OpenStack cloud is way more complex than deploying an application into an already working cloud because instead of being provided infrastructure that offers a higher level of abstraction, you must manage microservices, like Nova and Neutron, yourself. This results in managing many dependencies and keeping track of all of the areas that can fail because each microservice must work effectively with each other for the cloud to work properly.

When would a VM with Compose be sufficient, and what additional behavior did Kubernetes provide in your experiment?

VM's with Docker Compose would be sufficient if we only needed a single node in comparison to a multi-node application. Kubernetes also provides self-healing, which could be vital for keeping applications alive when things go wrong. If this is not required, container restarts with Docker Compose could suffice. Kubernetes also provides load balancing, which improves incoming traffic processing. If this is not needed, basic port mapping could be acceptable with Docker Compose.

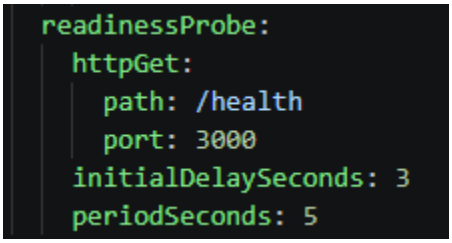
Additional Requirements for CS 554

1. Add CPU and memory requests and limits to the Kubernetes application container. Explain your choices and distinguish a request from a limit. Nova is the engine responsible for scheduling and managing the virtual machine instance. It receives creation requests, evaluates all available nodes, and downloads the selected image from Glance.



0.25 CPU cores and 64Mi memory for requests are typically efficient enough for a lightweight web server. To ensure we handle traffic spikes efficiently, setting the limits to double of the requests should suffice (0.50 cores and 128Mi).

2. Add a readiness probe using /health. Explain how readiness affects Service traffic and how it differs from restarting a container.



Every 5 seconds, Kubernetes hits the health/ endpoint. This increases the request count constantly. If the readiness probe ever fails, the Pod's IP is removed from the routing pool. Instead of restarting, the container remains running; but, no client is able to access it. If this was a liveness probe, the container would be restarted once the probe fails.

3. Perform a short readiness experiment in which one replica becomes NotReady while its application process continues running. One permitted approach is to temporarily alter the readiness probe on one Pod so it checks a non-serving path or port. Because Pods managed by a Deployment are recreated from its template, use a temporary per-Pod change (for example, a path to one running Pod's readiness proved when supported by your Kubernetes environment) or a clearly document application-level readiness toggle. Do not modify the shared Deployment template in a way that makes both replicas NotReady. Do not simply crash or delete the Pod: this is a readiness experiment, not a restart experiment. Capture a baseline (both Pods Ready and Service endpoints), the failure (one Pod running but NotReady and excluded from ready Service endpoints), and recovery (both Ready and eligible again). Include commands/configuration changes, Pod status, and EndpointSlice observations. If testing HTTP traffic, use an in-cluster client or a method that routes through ready Service backends. Port forwarding to a single Pod, including some Service port-forward sessions, does not establish traffic eligibility for all replicas. Restore any temporary changes after the experiment.

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl get pods
NAME                              READY   STATUS    RESTARTS   AGE
cs454-service-77b5d9cd84-szc2s    1/1     Running   0          12s
cs454-service-77b5d9cd84-wt6t6    1/1     Running   0          62s
```

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl get endpoints cs454-service
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME           ENDPOINTS          AGE
cs454-service  10.42.2.13:3000    130m
```

Baseline status for both pods in the cluster prior to altering any states

```
readinessProbe:
  exec:
    command:
    - sh
    - -c
    - '! test -f /tmp/fail-readiness'
  initialDelaySeconds: 3
  periodSeconds: 5
```

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl exec cs454-service-77b5d9cd84-szc2s -- sh -c "touch /tmp/fail-readiness"
```

Readiness Probe updates along with the commands to initiate specific pod readiness failure

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl get pods
NAME                              READY   STATUS    RESTARTS   AGE
cs454-service-77b5d9cd84-szc2s    0/1     Running   0          81s
cs454-service-77b5d9cd84-wt6t6    1/1     Running   0          2m11s
```

Results after pod updates

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl exec cs454-service-77b5d9cd84-szc2s -- sh -c "rm  /tmp/fail-readiness"
```

Command to restore specific pod's readiness probe condition

```
tanne@DESKTOP-S4VG2CO MINGW64 ~/OneDrive/Documents/GitHubProjects/CS554_CloudComputing/CS554-Midterm (main)
$ kubectl get pods
NAME                              READY   STATUS    RESTARTS   AGE
cs454-service-77b5d9cd84-szc2s    1/1     Running   0          12m
cs454-service-77b5d9cd84-wt6t6    1/1     Running   0          13m
```

Results after restoration

To satisfy the requirements of this specific readiness experiment, we needed to have both pods running, but only one in the ready state. An update to the deployment allowed me to successfully add a readiness toggle that could be applied to any chosen pod. Basically, it checks to see if a specific file (/temp/fail-readiness" exists in a pod. If it does, it fails the readiness check and if it does not, it passes. Initially, both pods remained in the ready state since neither of the pods contained that file. After adding

that file to an existing pod, that pod almost instantly switched to a not ready state. This resulted in all of the traffic being forwarded to the only remaining pod that is in the ready state. There were no restarting, replacing, or deleting pods during this experiment, just an automatic update via Kubernetes's orchestration capabilities when given a conditional. To restore the initial state of the cluster, all we needed to do was remove the fail-readiness file from the pod that was initially added to. This experiment represents the power of Kubernetes’s capabilities to keep traffic flowing for as long as there are resources to allocate to.